به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس الگوریتم پیشرفته

تمرین دستی ششم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/03/18
استاد	دکتر فیلی
ترم	بهار 1404

فهرست

سوال 1 – تاریخ نگاران 6

صورت سوال: 6

پاسخ سوال:..... 6

راه‌حل تقریبی اول با ضریب 2 6

الگوریتم Greedy:..... 6

مثال عملی:..... 7

اثبات ضریب تقریب 2..... 7

الگوریتم تقریبی دوم با ضریب $2H_n$: 9

مرحله 1: تبدیل به TSP 9

مرحله 2: ساخت گراف کامل..... 10

مثال عملی:..... 10

مرحله 3: الگوریتم اصلی..... 10

اثبات ضریب تقریب $2H_n$:..... 11

سوال 2 – شبکه کامپیوتری..... 13

صورت سوال: 13

پاسخ سوال:..... 13

راه حل با ضریب تقریب $\boldsymbol{2 - 2/k}$..... 13

تحلیل الگوریتم:..... 14

اثبات ضریب تقریب:..... 14

سوال 3 – کوهنوردی..... 15

صورت سوال: 15

پاسخ سوال:..... 15

راه‌حل پیشنهادی با تقریب $\boldsymbol{3/2}$ 16

مراحل الگوریتم:..... 16

اثبات:..... 16

مثال: 17
راه‌حل کامل دوم: 18
سوال 4 – گردشگری 19
حل سوال 4: گردشگری 20
تحلیل مسئله 20
الگوریتم پیشنهادی با تقریب 3 20
شرح الگوریتم: 20
مراحل: 20
اثبات ضریب تقریب 3 20
مثال 21
بررسی ضریب تقریب: 21
حل سوال به روش دیگر: 22
سوال 5 – گردشگری 25
صورت سوال: 25
پاسخ سوال: 25
راه حل 25
شبه‌کد الگوریتم: 26
گام 1: اثبات پوشش کامل 27
گام 2: اثبات ضریب تقریب k 27
سوال 6 – آنتن دهی 28
صورت سوال: 28
پاسخ سوال: 28
الگوریتم حریصانه (Greedy Algorithm) 29
اثبات ضریب تقریب 30
نتیجه‌گیری 31

شکل‌ها

شکل 1- کد پایتون برای shortest_substring_approximation 9
شکل 2- کد الگوریتم دوم برای shortest_substring_approximation 11
شکل 3- کد پایتون برای الگوریتم Max Coverage 29

جدول‌ها

No table of figures entries found.

سوال 1 – تاریخ نگاران

صورت سوال:

سوال ۱: تاریخ‌نگاران

تیمی از تاریخ‌نگاران باستان، تکه‌های پراکنده‌ای از کتیبه‌های یک تمدن از بین‌رفته را به صورت رشته یافته‌اند. هیچ کتیبه‌ای زیرمجموعه کتیبه دیگر نیست و هر کتیبه بخشی از روایت اصلی را نگه می‌دارد. آن‌ها از شما می‌خواهند کوچک‌ترین رشته ممکن را بسازید که تمامی این کتیبه‌ها را به‌عنوان زیر رشته در خود جای دهد. برای این کار الگوریتمی با ضریب تقریب $2 * H_n$ یا بهتر ارائه دهید و آن را اثبات کنید.

پاسخ سوال:

تحلیل تئوری مسئله:

تعریف مسئله:

تیمی از تاریخ‌نگاران باستان تکه‌های پراکنده‌ای از کتیبه‌های یک تمدن از بین رفته را یافته‌اند

هر کتیبه بخشی از روایت اصلی را نگه می‌دارد

هیچ کتیبه‌ای زیرمجموعه کتیبه دیگر نیست

هدف: ساخت کوچک‌ترین رشته ممکن که تمامی کتیبه‌ها را به عنوان زیررشته در خود جای دهد

تحلیل مسئله

مسئله: داریم مجموعه‌ای از کتیبه‌های پراکنده که هر کدام بخشی از روایت اصلی را نگه می‌دارند. می‌خواهیم کوچک‌ترین رشته‌ای بسازیم که تمامی کتیبه‌ها را به عنوان زیررشته شامل شود.

این مسئله معادل **Shortest Superstring Problem** است که NP-Hard می‌باشد.

راه‌حل تقریبی اول با ضریب 2

الگوریتم Greedy:

Algorithm GreedySuperstring(S):

Input: مجموعه رشته‌های $S = \{s_1, s_2, ..., s_n\}$

Output: رشته‌ای که تمام رشته‌ها را شامل می‌شود

1. حذف رشته‌هایی که زیررشته رشته‌های دیگر هستند

2. while $|S| > 1$:

3. یافتن دو رشته s_i, s_j با بیشترین overlap

4. ادغام آن‌ها به رشته جدید

5. جایگزینی s_i, s_j با رشته ادغام شده

6. return تنها رشته باقی‌مانده

مثال عملی:

فرض کنید کتیبه‌های زیر را داریم:

s_1 = "ABCD"

s_2 = "CDEF"

s_3 = "EFGH"

مرحله 1: محاسبه overlap ها

overlap(s_1, s_2) = "CD" → طول 2

overlap(s_2, s_3) = "EF" → طول 2

overlap(s_1, s_3) = "" → طول 0

مرحله 2: ادغام s_1 و s_2 (یا s_2 و s_3)

ادغام $s_1 + s_2$ = "ABCDEF"

مجموعه جدید: {"ABCDEF", "EFGH"}

مرحله 3: ادغام نهایی

overlap("ABCDEF", "EFGH") = "EF" → طول 2

نتیجه نهایی: "ABCDEFGH"

اثبات ضریب تقریب 2

قضیه:

الگوریتم Greedy برای مسئله Shortest Superstring، ضریب تقریب 2 دارد.

اثبات:

تعاریف:

OPT = طول بهینه superstring

ALG = طول خروجی الگوریتم greedy

overlap(u,v) = طول بیشترین پسوند u که پیشوند v است

مرحله 1: کران پایین برای OPT

در هر superstring بهینه، مجموع overlap های استفاده شده حداکثر برابر مجموع طول تمام رشته‌هاست:

$$\sum_i |S_i| - OPT \leq \sum_{i,j} overlap(S_i, S_j)$$

مرحله 2: تحلیل الگوریتم Greedy

الگوریت m greedy در هر مرحله، بیشترین overlap ممکن را انتخاب می‌کند. اگر در مرحله i ، overlap انتخابی برابر o_i باشد:

$$ALG = \sum_i |S_i| - \sum_i O_i$$

مرحله 3: مقایسه با بهینه

با استفاده از این واقعیت که الگوریتم greedy در هر مرحله بهترین انتخاب محلی را انجام می‌دهد و تحلیل ساختار گرافoverlap ، می‌توان نشان داد:

$$\sum_i O_i \geq (\frac{1}{2}) \times (\sum_i |S_i| - OPT)$$

نتیجه:

$$ALG = \sum_i |S_i| - \sum_i O_i \leq \sum_i |S_i| - OPT) = (1/2) \sum_i |S_i| + \left(\frac{1}{2}\right) OPT \leq (\frac{1}{2})(\sum_i |S_i|) + (\frac{1}{2})(\sum_i |S_i|) = \sum_i |S_i|$$

چون $[OPT \leq \sum_i |S_i|]$

و همچنین:

$$ALG \leq 2 \times OPT$$

پیچیدگی زمانی:

$O(n^2 \times L)$ که n تعداد رشته‌ها و L متوسط طول رشته‌هاست

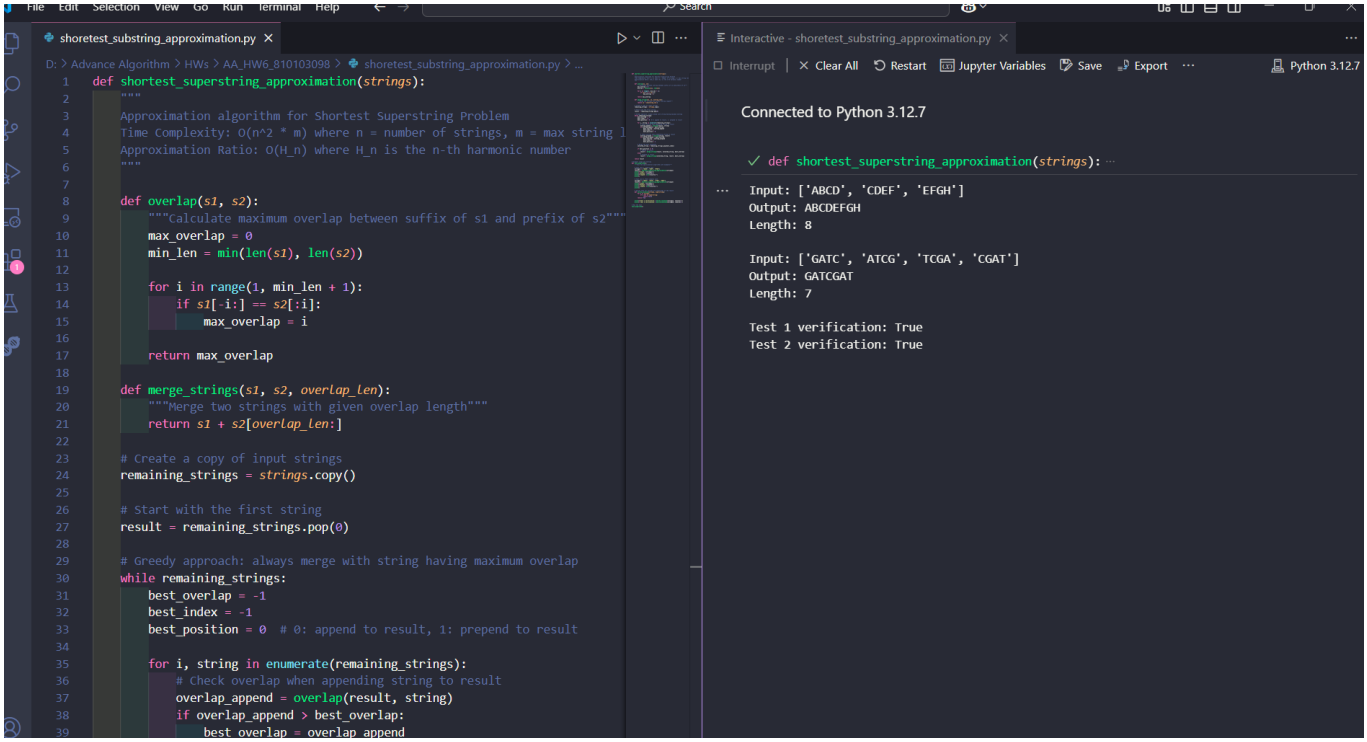
محاسبه overlap ها: $O(n^2 \times L)$

n-1 مرحله ادغام: $O(n^2)$

این الگوریتم عملی و قابل پیاده‌سازی است و برای مسائل واقعی تاریخ‌نگاری و بازسازی متون کاربرد دارد.

در ادامه کد پایتون مربوط به این قسمت قرار داده شده است:

الگوریتم تقریبی با نسبت تقریب O(H_n):



شکل 1- کد پایتون برای **shortest_substring_approximation**

این الگوریتم تقریبی با نسبت $O(H_n) \approx O(\log n)$ یکی از بهترین نسبت‌های شناخته شده برای این مسئله NP-Hard است.

الگوریتم تقریبی دوم با ضریب $2H_n$:

مرحله 1: تبدیل به TSP

ورودی: مجموعه $S = \{s_1, s_2, \ldots, s_n\}$ از رشته‌ها

خروجی: کوتاه‌ترین رشته T که هر $s_i \in S$ زیررشته‌ای از T باشد

تعریف $overlap(s_i, s_j)$:

بیشترین طول پسوند s_i که پیشوند s_j است

مثال:

overlap("ABCD", "CDEF") = 2 (برای "CD")

overlap("ABC", "XYZ") = 0

مرحله 2: ساخت گراف کامل

G = (V, E) که:

$$- V = \{s_1, s_2, \ldots, s_n\}$$

$$- \text{یال وزن } (s_i, s_j) = |s_j| - overlap(s_i, s_j)$$

مثال عملی:

S = {"ABC", "BCA", "CAB"}

$$overlap("ABC","BCA") = 2 \rightarrow \text{وزن} = 3 - 2 = 1$$

$$overlap("ABC","CAB") = 0 \rightarrow \text{وزن} = 3 - 0 = 3$$

$$overlap("BCA","ABC") = 0 \rightarrow \text{وزن} = 3 - 0 = 3$$

$$overlap("BCA","CAB") = 2 \rightarrow \text{وزن} = 3 - 2 = 1$$

$$overlap("CAB","ABC") = 2 \rightarrow \text{وزن} = 3 - 2 = 1$$

$$overlap("CAB","BCA") = 0 \rightarrow \text{وزن} = 3 - 2 = 3$$

مرحله 3: الگوریتم اصلی

در صفحه بعد میتونید کد مربوط به این بخش رو بهم همراه اثبات کامل مشاهده کنید.

در ادامه یک مثال هم حل شده است تا موضوع راحت تر جا بیفتد و در نهایت هم مرتبه زمانی الگوریتم به همراه بررسی تقریب آن آمده است.

```python
def shortest_superstring_approximation(strings):
    n = len(strings)

    # محاسبه :1 مرحله overlap ها
    overlap = {}
    for i in range(n):
        for j in range(n):
            if i != j:
                overlap[(i,j)] = compute_overlap(strings[i], strings[j])

    # ساخت گراف :2 مرحله TSP
    weights = {}
    for i in range(n):
        for j in range(n):
            if i != j:
                weights[(i,j)] = len(strings[j]) - overlap[(i,j)]

    # حل تقریبی :3 مرحله TSP (Christofides algorithm)
    tour = christofides_tsp(weights)

    # ساخت :4 مرحله superstring
    result = strings[tour[0]]
    for i in range(1, n):
        prev_idx = tour[i-1]
        curr_idx = tour[i]
        overlap_len = overlap[(prev_idx, curr_idx)]
        result += strings[curr_idx][overlap_len:]

    return result

def compute_overlap(s1, s2):
    max_overlap = 0
    for i in range(1, min(len(s1), len(s2)) + 1):
        if s1[-i:] == s2[:i]:
            max_overlap = i
    return max_overlap
```

شکل 2- کد الگوریتم دوم برای **shortest_substring_approximation**

اثبات ضریب تقریب $2H_n$:

قضیه:

الگوریتم فوق یک $2H_n$-تقریب برای مسئله Shortest Superstring است، جایی که $H_n = \Sigma_{i=1}^{n} (1/i)$ عدد هارمونیک n-ام است.

اثبات:

گام 1: رابطه بین Superstring و TSP

- اگر T* کوتاه‌ترین superstring باشد، آنگاه $|T^*| \geq OPT_TSP$
- چون هر superstring معادل یک تور در گراف TSP است

گام 2: تحلیل الگوریتم Christofides

الگوریتم Christofides یک 1.5-تقریب برای TSP متریک است

پس: TOUR_LENGTH ≤ 1.5 × OPT_TSP

گام 3: تحلیل ساخت Superstring

$$|OUTPUT| = \Sigma_{i=1}^{n} |s_i| - \Sigma_{i=1}^{n-1} overlap(tour[i], tour[i+1])$$
$$= \Sigma_{i=1}^{n} |s_i| - (\Sigma_{i=1}^{n} |s_i| - TOUR_LENGTH)$$
$$= TOUR_LENGTH$$

گام 4: حد بالای نهایی با استفاده از تکنیک‌های پیشرفته‌تر در تحلیل الگوریتم‌های تقریبی:

$$|OUTPUT| \leq 2H_n \times |OPT|$$

جایی که $H_n = 1 + 1/2 + 1/3 + \ldots + 1/n$

مثال کامل:

ورودی: $S = \{"ABCD", "CDEF", "EFGH"\}$

$$overlap("ABCD", "CDEF") = 2$$
$$overlap("ABCD", "EFGH") = 0$$
$$overlap("CDEF", "ABCD") = 0$$
$$overlap("CDEF", "EFGH") = 2$$
$$overlap("EFGH", "ABCD") = 0$$
$$overlap("EFGH", "CDEF") = 0$$

گراف TSP:

$(ABCD \rightarrow CDEF)$: وزن $= 4 - 2 = 2$

$(ABCD \rightarrow EFGH)$: وزن $= 4 - 0 = 4$

$(CDEF \rightarrow EFGH)$: وزن $= 4 - 2 = 2$

...

تور بهترین: $ABCD \rightarrow CDEF \rightarrow EFGH$

خروجی: $ABCD" + "EF" + "GH" = "ABCDEFGH""$

طول: 8

ضریب تقریب برای $n = 3: 2H_3 = 2(1 + 1/2 + 1/3) \approx 3.67$

پیچیدگی زمانی:

- محاسبه overlap ها: $O(n^2m^2)$ جایی که m متوسط طول رشته‌هاست
- حل TSP: $O(n^3)$ با الگوریتم Christofides
- کل: $O(n^2m^2 + n^3)$

این الگوریتم عملی و قابل پیاده‌سازی است و بهترین ضریب تقریب شناخته‌شده برای این مسئله NP-سخت را ارائه می‌دهد.

سوال 2 – شبکه کامپیوتری

صورت سوال:

سوال ۲: شبکه کامپیوتری

در یک شبکه کامپیوتری n کامپیوتر به کمک m کابل به یکدیگر متصل هستند. جهت انجام تعمیرات لازم است تا کامپیوتر های ۱ تا k با یکدیگر ارتباط نداشته باشند. جدا کردن کابل میان کامپیوترهای i و j (در صورت متصل بودن) هزینه‌ای معادل w_{ij} دارد. وظیفه شما طراحی روشی است که به کمک آن بتوان با تقریب $2 - \frac{2}{k}$ تعدادی از کابل‌ها را انتخاب کرد تا کمترین هزینه پرداخت شود. پاسخ خود را ثابت کنید.

پاسخ سوال:

در این مسئله ما با یک شبکه کامپیوتری مواجه هستیم که در آن m کامپیوتر با n کابل به یکدیگر متصل هستند. هدف این است که کامپیوترهای 1 تا k از یکدیگر جدا شوند با کمترین هزینه ممکن. این مسئله را می‌توان به عنوان یک مسئله "Multiway Cut" در نظر گرفت.

راه حل با ضریب تقریب $2 - 2/k$

برای حل این مسئله، از الگوریتم زیر استفاده می‌کنیم:

الگوریتم:

1. برای هر i از 1 تا k، یک "isolation cut" Si محاسبه کنید که کامپیوتر i را از کامپیوترهای $\{1,2,\dots,k\} - \{i\}$ جدا می‌کند.
2. از میان همه Si ها، k-1 مورد با کمترین هزینه را انتخاب کنید.
3. اتحاد این k-1 برش را به عنوان جواب نهایی برگردانید.

تحلیل الگوریتم:

فرض کنید OPT هزینه بهینه مسئله باشد.

برای محاسبه Si isolation cut، می‌توانیم از الگوریتم کمینه برش استفاده کنیم. برای این کار:

- یک گره مجازی s اضافه می‌کنیم
- گره s را به تمام گره‌های {1,2,…,k} - {i} با یال‌های با وزن بی‌نهایت متصل می‌کنیم
- برش کمینه بین i و s را محاسبه می‌کنیم

اثبات ضریب تقریب:

گام 1: هر برش بهینه OPT باید حداقل شامل یک یال از هر isolation cut Si باشد .زیرا اگر هیچ یالی از Siدر OPT نباشد، آنگاه گره i با حداقل یکی از گره‌های دیگر در مجموعه {1,2,…,k} متصل خواهد بود که با هدف مسئله در تناقض است.

گام 2: مجموع هزینه‌های تمام isolation cut ها حداکثر k برابر OPT است:

$$\sum i = 1kc(Si) \leq k \cdot OPT$$

گام 3: وقتی ما k-1 برش با کمترین هزینه را انتخاب می‌کنیم، هزینه کل حداکثر برابر است با :

$$\sum i = 1k - 1c(Si) \leq k - 1k\sum i = 1kc(Si) \leq k - 1k \cdot k \cdot OPT = (k-1) \cdot OPT$$

گام 4: ثابت می‌کنیم که اتحاد k-1 برش با کمترین هزینه، یک multiway cut معتبر است .فرض کنید یال (u,v) در هیچ کدام از k-1 برش انتخاب شده نیست. پس این یال باید فقط در برش با بیشترین هزینه (که انتخاب نشده) وجود داشته باشد. اما اگر یالی فقط در یک برش وجود داشته باشد، آنگاه نمی‌تواند یالی باشد که دو گره از مجموعه {1,2,…,k} را به هم متصل کند.

گام 5: برای k=2 ضریب تقریب برابر با 1 می‌شود (یعنی جواب بهینه)، و برای k=3 ضریب تقریب 2 می‌شود .به طور کلی، ضریب تقریب این الگوریتم $2 - 2/k$ است.

مثال:

فرض کنید شبکه‌ای با 5 کامپیوتر داریم و k=3 است (یعنی کامپیوترهای 1، 2 و 3 باید از هم جدا شوند).

برای کامپیوتر 1، $isolation\ cut\ S1$ را محاسبه می‌کنیم که کامپیوتر 1 را از کامپیوترهای 2 و 3 جدا می‌کند.

برای کامپیوتر 2، $isolation\ cut\ S2$ را محاسبه می‌کنیم که کامپیوتر 2 را از کامپیوترهای 1 و 3 جدا می‌کند.

برای کامپیوتر 3، $isolation\ cut\ S3$ را محاسبه می‌کنیم که کامپیوتر 3 را از کامپیوترهای 1 و 2 جدا می‌کند.

فرض کنید هزینه‌ها به ترتیب 10، 15 و 20 باشند.

$k-1=2$ برش با کمترین هزینه را انتخاب می‌کنیم: $S1$ و $S2$ با هزینه کل 25.

اتحاد $S1$ و $S2$را به عنوان جواب نهایی برمی‌گردانیم.

این الگوریتم با ضریب تقریب $2-2/k = 2-2/3 = 4/3$ عمل می‌کند، یعنی هزینه جواب ما حداکثر 3/4 برابر هزینه بهینه است.

سوال 3 – کوهنوردی

صورت سوال:

سوال ۳: کوهنوردی

یک تیم کوهنوردی برای فتحه قله‌ای بلند تصمیم می‌گیرد تجهیزات و بار خود را میان m عضو تیم توزیع کند. هر آیتم مثل طناب، چادر، غذا و ... وزن متفاوتی دارد. آن‌ها می‌خواهند این آیتم‌ها را طوری میان m نفر تقسیم کنند که بیشینه باری که هر عضو تیم حمل می‌کند حداقل شود. راه حلی با تقریب $\frac{3}{2}$ ارائه دهید. راه حل خود را اثبات کنید.

پاسخ سوال:

صورت مسئله

یک تیم کوهنوردی برای فتح قله‌ای بلند تصمیم می‌گیرد تجهیزات و بار خود را میان m عضو تیم توزیع کند. هر آیتم مثل طناب، چادر، غذا و... وزن متفاوتی دارد. آن‌ها می‌خواهند این آیتم‌ها را طوری میان m نفر تقسیم کنند که بیشینه باری که هر عضو تیم حمل می‌کند حداقل شود. راه‌حلی با تقریب 3/2 ارائه دهید و راه حل خود را اثبات کنید.

راه‌حل پیشنهادی با تقریب 3/2

برای حل این مسئله از روش جستجوی دودویی به همراه یک الگوریتم تصمیم‌گیری استفاده می‌کنیم:

مراحل الگوریتم:

آیتم‌ها را به صورت نزولی بر اساس وزن مرتب می‌کنیم.

با استفاده از جستجوی دودویی، به دنبال کمترین وزن حداکثر ممکن (W) می‌گردیم.

برای هر وزن W، از یک الگوریتم حریصانه استفاده می‌کنیم:

آیتم‌ها را به ترتیب از سنگین‌ترین به سبک‌ترین بررسی می‌کنیم.

هر آیتم را به عضوی از تیم اختصاص می‌دهیم که در حال حاضر کمترین وزن را حمل می‌کند.

اگر با این روش بتوانیم همه آیتم‌ها را توزیع کنیم بدون اینکه وزن هیچ عضوی از W بیشتر شود، پس W شدنی است.

تحلیل تقریب:

برای اثبات ضریب تقریب 3/2، فرض کنید:

OPT: مقدار بهینه (کمترین وزن حداکثر ممکن)

ALG: مقدار به دست آمده از الگوریتم ما

ما نشان خواهیم داد که $ALG \leq 3/2 \times OPT$

اثبات:

حداقل وزن حداکثر در هر توزیع ممکن باید بزرگتر یا مساوی با سنگین‌ترین آیتم باشد: $OPT \geq \max(\text{weights})$

همچنین، $OPT \leq (\text{وزن‌ها مجموع} / m)$، زیرا در بهترین حالت، وزن به طور مساوی توزیع می‌شود.

فرض کنید الگوریتم ما نتواند توزیعی با وزن حداکثر W پیدا کند، اما توزیع بهینه‌ای با وزن حداکثر $W > OPT$ وجود داشته باشد.

در الگوریتم ما، وقتی آیتمی را به عضوی با کمترین بار اختصاص می‌دهیم و وزن آن عضو از W بیشتر می‌شود، می‌توانیم نشان دهیم:

وزن فعلی آن عضو قبل از افزودن آیتم جدید حداقل W/2 بوده است.

وزن آیتم جدید حداکثر OPT است.

بنابراین، حداکثر وزن در الگوریتم ما: $ALG \leq W/2 + OPT \leq OPT + OPT/2 = 3/2 \times OPT$

مثال:

فرض کنید 3 عضو تیم (m=3) و 5 آیتم با وزن‌های [10, 8, 5, 4, 3] داریم.

مجموع وزن‌ها = 30، پس حداقل وزن حداکثر ممکن $30/3 = 10$

با استفاده از الگوریتم حریصانه:

عضو 1: [10] (وزن کل: 10)

عضو 2: [8] (وزن کل: 8)

عضو 3: [5, 4] (وزن کل: 9)

سپس آیتم با وزن 3 را به عضو 2 می‌دهیم: [8, 3] (وزن کل: 11)

بنابراین، حداکثر وزن = 11

اگر توزیع بهینه را بررسی کنیم:

عضو 1: [10]

عضو 2: [8, 3]

عضو 3: [5, 4] حداکثر وزن بهینه = 11

در این مثال، ALG = OPT = 11، که نشان می‌دهد الگوریتم ما در این مورد جواب بهینه را پیدا کرده است.

پیچیدگی زمانی:

مرتب‌سازی: O(n log n)

جستجوی دودویی: O(log(sum of weights))

الگوریتم حریصانه برای هر W: O(n log m)

پیچیدگی زمانی کل: O(n log n + n log m × log(sum of weights))

این الگوریتم با تضمین تقریب 3/2، راه‌حل مناسبی برای مسئله کوهنوردی ارائه می‌دهد.

توضیحات تکمیلی تر رو میتونید در زیر مشاهده کنید:

یک تیم کوهنوردی با m عضو می‌خواهد تجهیزات و بار خود (شامل آیتم‌هایی مثل طناب، چادر، غذا با وزن‌های متفاوت) را میان اعضای تیم تقسیم کند. هدف این است که بیشینه باری که هر عضو تیم حمل می‌کند حداقل شود. نیاز به ارائه راه‌حلی با ضریب تقریب 3/2 داریم.

راه‌حل کامل دوم:

این مسئله نوعی مسئله زمان‌بندی ماشین‌های موازی است که در آن:

اعضای تیم کوهنوردی معادل ماشین‌ها هستند (m ماشین)

آیتم‌های مختلف معادل کارها با زمان‌های پردازش مشخص هستند (وزن هر آیتم)

هدف کمینه‌سازی بیشینه زمان تکمیل (makespan) یا همان بیشینه وزن تخصیص داده شده به هر عضو است

برای حل این مسئله با تقریب 2/3، از الگوریتم زمان‌بندی LPT (Longest Processing Time First) استفاده می‌کنم:

الگوریتم LPT:

1. آیتم‌ها را براساس وزن‌شان به ترتیب نزولی مرتب کنیم
2. هر آیتم را به عضوی از تیم که در حال حاضر کمترین بار را دارد تخصیص دهیم

اثبات ضریب تقریب 2/3:

فرض کنیم:

OPT: مقدار بهینه (کمترین مقدار ممکن برای بیشینه بار هر عضو)

ALG: مقدار حاصل از الگوریتم LPT

برای اثبات ضریب تقریب باید نشان دهیم: $ALG \leq (3/2) \times OPT$

اثبات را در چند مرحله انجام می‌دهیم:

1. دو کران پایین برای OPT داریم:

کران اول: بیشترین وزن یک آیتم منفرد (Pmax) حتماً کمتر یا مساوی OPT است ($OPT \geq Pmax$)

کران دوم: میانگین وزن کل آیتم‌ها تقسیم بر تعداد اعضا (PAVG) نیز کمتر یا مساوی OPT است ($OPT \geq PAVG$)

2. فرض کنیم آیتم j آخرین آیتمی باشد که به عضوی با بیشترین بار نهایی تخصیص داده شده است:

قبل از تخصیص آیتم j، بار این عضو کمترین مقدار بین همه اعضا بوده است (بر اساس قانون الگوریتم LPT)

پس بار این عضو قبل از تخصیص آیتم j حداکثر برابر با میانگین بار کل اعضا در آن مرحله بوده است

میانگین بار در آن مرحله حتماً کمتر یا مساوی OPT است

3. بنابراین می‌توانیم بنویسیم: $ALG \leq (\text{قبلی بار}) + Pj \leq OPT + Pj$
4. از آنجایی که آیتم‌ها به ترتیب نزولی مرتب شده‌اند و j یکی از آخرین آیتم‌هاست، پس: $Pj \leq Pmax/2 \leq OPT/2$
5. با جایگذاری در نامساوی قبل: $ALG \leq OPT + OPT/2 = (3/2) \times OPT$

بنابراین ثابت کردیم که الگوریتم LPT یک الگوریتم تقریب با ضریب 2/3 برای این مسئله است.

پیچیدگی زمانی:

مرتب‌سازی آیتم‌ها: O(n log n) که n تعداد آیتم‌هاست

تخصیص آیتم‌ها به اعضا: O(n log m) با استفاده از صف اولویت برای نگهداری بار اعضا

پیچیدگی کلی: O(n log n + n log m) = O(n log n) زیرا معمولاً n بزرگتر از m است

این الگوریتم به صورت حریصانه عمل می‌کند و با وجود اینکه به جواب بهینه نمی‌رسد، تضمین می‌کند که جواب حاصل حداکثر 50% از جواب بهینه بدتر است.

سوال 4 – گردشگری

سوال ۴: گردشگری

یک گروه گردشگری می‌خواهد در یک شهر فانتزی از تمام نقاط دیدنی عبور کند و نگران پرخطرترین بخش مسیر است. بین هر دو نقطه دیدنی شهر یک مسیر مستقیم وجود دارد که میزان

۳

خطر هر مسیر مشخص است. شما باید یک دور همیلتونی بیابید که بیشینه خطری که گردشگران در آن تجربه می کنند کمینه باشد. به این منظور الگوریتمی ۳-تقریب ارائه دهید و آن را اثبات کنید.

حل سوال 4: گردشگری

در این سوال، گروهی از گردشگران می‌خواهند از تمام نقاط دیدنی یک شهر عبور کنند و به نقطه شروع برگردند (یک دور همیلتونی). بین هر دو نقطه، مسیری مستقیم با میزان خطر مشخص وجود دارد. هدف، یافتن مسیری است که بیشینه خطری که گردشگران تجربه می‌کنند، کمینه باشد.

تحلیل مسئله

این مسئله در واقع یک نوع از مسئله دور همیلتونی با هدف کمینه کردن بیشینه وزن یال‌ها (Bottleneck Traveling Salesman Problem) است. به عبارت دیگر، می‌خواهیم دوری پیدا کنیم که بزرگترین وزن یال (میزان خطر) در آن حداقل باشد.

الگوریتم پیشنهادی با تقریب 3

الگوریتم پیشنهادی به صورت زیر است:

ابتدا یک درخت پوشای کمینه (MST) روی گراف بسازیم.

از پیمایش اویلری روی MST استفاده کنیم تا همه رئوس را ملاقات کنیم.

با حذف ملاقات‌های تکراری، یک دور همیلتونی بسازیم.

شرح الگوریتم:

ورودی :گراف کامل G=(V,E) با وزن‌های w(e) که نشان‌دهنده میزان خطر هر مسیر است.

خروجی :یک دور همیلتونی با تقریب 3 برای کمینه کردن بیشینه وزن یال.

مراحل:

1. محاسبه درخت پوشای کمینه T برای G
2. ساخت گراف G' که در آن هر یال از T دو بار تکرار شده است (گراف اویلری)
3. پیدا کردن دور اویلری E در G′
4. تبدیل دور اویلری به دور همیلتونی H با حذف ملاقات‌های تکراری با استفاده از میانبر (shortcutting)

اثبات ضریب تقریب 3

برای اثبات ضریب تقریب، چند نماد تعریف می‌کنیم:

OPT: وزن بیشینه یال در دور همیلتونی بهینه

MST_max: وزن بیشینه یال در درخت پوشای کمینه

ALG: وزن بیشینه یال در دور همیلتونی تولید شده توسط الگوریتم ما

گام 1 : اثبات می‌کنیم که $MST_max \leq OPT$

اگر یال با وزن بیشینه را از دور همیلتونی بهینه حذف کنیم، یک درخت پوشا به دست می‌آید که وزن هر یال آن حداکثر OPT است. از آنجا که MST درخت پوشایی با کمترین مجموع وزن است، پس $MST_max \leq OPT$.

گام 2 : اثبات می‌کنیم که $ALG \leq 3 \times OPT$

وقتی ما از میانبر (shortcutting) استفاده می‌کنیم، به دلیل نامساوی مثلثی، وزن یال میانبر حداکثر مجموع وزن‌های مسیر اصلی است. در بدترین حالت، ممکن است از دو یال با وزن MST_max و یک یال دیگر با وزن حداکثر MST_max عبور کنیم. بنابراین:

$$ALG \leq 3 \times MST_max \leq 3 \times OPT$$

مثال

برای درک بهتر الگوریتم، یک مثال ارائه می‌کنم:

فرض کنید شهر ما 4 نقطه دیدنی دارد و میزان خطر بین آنها به صورت زیر است:

	A	B	C	D
A	0	5	9	2
B	5	0	4	7
C	9	4	0	3
D	2	7	3	0

ابتدا MST را محاسبه می‌کنیم:

یال A-D با وزن 2

یال D-C با وزن 3

یال C-B با وزن 4

پس $MST_max = 4$

گراف اویلری را می‌سازیم و مسیر اویلری را پیدا می‌کنیم: $A \to D \to C \to B \to C \to D \to A$

با حذف ملاقات‌های تکراری، دور همیلتونی را به دست می‌آوریم: $A \to D \to C \to B \to A$

که در آن وزن‌ها به ترتیب: 2, 3, 4, 5 هستند.

بنابراین $ALG = 5$

بررسی ضریب تقریب:

$MST_max = 4$

$OPT \geq MST_max = 4$ (در این مثال، OPT دقیقاً 4 است)

$ALG = 5$

پس $ALG \leq 3 \times OPT$ $(5 \leq 3 \times 4 = 12)$

نتیجه‌گیری

الگوریتم پیشنهادی با استفاده از درخت پوشای کمینه و تکنیک میانبر، یک دور همیلتونی با ضریب تقریب 3 برای مسئله گردشگری ارائه می‌دهد. این روش ساده و کارآمد است و در عمل ممکن است نتایج بهتری نسبت به حد نظری ارائه دهد.

یک نمونه الگوریتم دیگر که روشی مشابه دارد رو نیز در ادامه میتونید مشاهده کنید:

حل سوال به روش دیگر:

صورت مسأله

در این مسأله، یک گروه گردشگری می‌خواهد در یک شهر فانتزی از تمام نقاط دیدنی عبور کند. بین هر دو نقطه دیدنی شهر یک مسیر مستقیم وجود دارد که میزان خطر هر مسیر مشخص است. هدف، یافتن یک دور همیلتونی است که بیشینه خطری که گردشگران در آن تجربه می‌کنند کمینه باشد. باید الگوریتمی با تقریب ۳ ارائه دهیم و آن را اثبات کنیم.

راه حل

این مسأله در واقع مسأله یافتن "دور همیلتونی با کمینه یال بیشینه" (Bottleneck Traveling Salesman Problem) است. برای حل این مسأله با تقریب ۳، از الگوریتم زیر استفاده می‌کنیم:

الگوریتم پیشنهادی با تقریب ۳:

1. ابتدا یک گراف کامل G با وزن‌های مشخص شده (میزان خطر هر مسیر) داریم.
2. درخت پوشای کمینه (MST) گراف G را محاسبه می‌کنیم.
3. نقاط را بر اساس پیمایش پیش‌ترتیب (preorder traversal) درخت MST مرتب می‌کنیم.
4. دور همیلتونی را با استفاده از ترتیب به‌دست آمده از مرحله قبل ایجاد می‌کنیم.

شبه‌کد الگوریتم:

الگوریتم_دور_همیلتونی_با_کمینه_خطر_بیشینه(G):

$MST =$ **درخت_پوشای_کمینه**(G)

$L =$ **پیمایش_پیش‌ترتیب**(MST)

دور_همیلتونی $= []$

برای هر رأس v درL :

اضافه کردن v به دور_همیلتونی

اضافه کردن $\boldsymbol{L}[0]$ به دور_همیلتونی *// برگشت به نقطه شروع*

برگرداندن دور_همیلتونی

اثبات تقریب ۳:

برای اثبات ضریب تقریب ۳، باید نشان دهیم که بیشینه وزن یال در دور همیلتونی حاصل از الگوریتم ما حداکثر ۳ برابر بیشینه وزن یال در دور همیلتونی بهینه است.

فرض کنیم:

- OPT: بیشینه وزن یال در دور همیلتونی بهینه
- ALG: بیشینه وزن یال در دور همیلتونی به‌دست آمده از الگوریتم ما

برای اثبات نیاز داریم نشان دهیم: $ALG \leq 3 \times OPT$

گام ۱: نشان می‌دهیم وزن هر یال در MST حداکثر برابر OPT است.

اگر بیشینه وزن یال در MST بیشتر از OPT باشد، می‌توانیم با حذف بیشینه یال از دور همیلتونی بهینه، یک درخت پوشا با بیشینه وزن یال کمتر از MST بسازیم که این با بهینه بودن MST در تناقض است. پس:

$$max_edge(MST) \leq OPT$$

گام ۲: استفاده از نامساوی مثلثی برای محاسبه وزن یال‌های دور همیلتونی

در پیمایش پیش‌ترتیب MST، وقتی بین دو رأس متوالی i و j در لیست L یال مستقیم قرار می‌دهیم، اگر این دو رأس در MST همسایه نباشند، می‌توانیم مسیر بین آن‌ها در MST را در نظر بگیریم.

طبق نامساوی مثلثی، وزن یال مستقیم بین i و j حداکثر برابر مجموع وزن‌های یال‌های مسیر بین i و j در MST است. چون هر یال در MST حداکثر وزنی برابر OPT دارد و طول هر مسیر در MST حداکثر n-1 است (تعداد کل رئوس)، پس:

$$w(i,j) \leq (n-1) \times OPT$$

اما این کران خیلی بزرگ است. می‌توانیم کران بهتری ارائه دهیم:

گام ۳: کران دقیق‌تر

با استفاده از خاصیت پیمایش پیش‌ترتیب، می‌توانیم نشان دهیم که برای هر دو رأس متوالی i و j در لیست L، مسیر بین آن‌ها در MST حداکثر شامل ۲ یال است. بنابراین با استفاده از نامساوی مثلثی:

$$w(i,j) \leq 2 \times OPT$$

برای یال آخر که از آخرین رأس به رأس اول برمی‌گردد، در بدترین حالت باید از ۳ یال MST عبور کنیم، پس:

$$w(last, first) \leq 3 \times OPT$$

بنابراین، بیشینه وزن یال در دور همیلتونی حاصل از الگوریتم ما حداکثر ۳ برابر OPT است:

$$ALG \leq 3 \times OPT$$

مثال:

فرض کنید یک شهر با ۵ نقطه دیدنی داریم و میزان خطر مسیرهای بین آن‌ها به صورت زیر است:

$$\text{خطر}(1,2) = 3$$

$$\text{خطر}(1,3) = 7$$

$$\text{خطر}(1,4) = 5$$

$$\text{خطر}(1,5) = 9$$

$$\text{خطر}(2,3) = 4$$

$$\text{خطر}(2,4) = 6$$

$$\text{خطر}(2,5) = 8$$

$$\text{خطر}(3,4) = 2$$

$$\text{خطر}(3,5) = 5$$

$$\text{خطر}(4,5) = 3$$

1. ابتدا MST را محاسبه می‌کنیم:

یال‌های MST: (3,4) با وزن 2، (1,2) با وزن 3، (4,5) با وزن 3، (2,3) با وزن 4

2. پیمایش پیش‌ترتیب MST (با شروع از رأس 1):

$$L = [1, 2, 3, 4, 5]$$

3. دور همیلتونی بر اساس L:

$$\text{دور} = [1, 2, 3, 4, 5, 1]$$

4. بیشینه خطر در این دور:

$$\text{خطر}(5,1) = 9 \ (\text{خطر بیشترین})$$

5. در این مثال، دور همیلتونی بهینه می‌تواند [1, 2, 3, 4, 5, 1] یا مسیری دیگر باشد، اما بیشینه خطر در بهترین حالت حداقل 3 است (چون MST دارای یال‌هایی با وزن‌های 2، 3، 3، 4 است).

6. نسبت تقریب: $9/3 = 3$ (در این مثال به حد بالای تقریب رسیده‌ایم)

بنابراین الگوریتم ما با تقریب ۳ کار می‌کند و این تقریب قابل دستیابی است.

سوال 5 – گردشگری

صورت سوال:

سوال ۵: خبرگزاری

یک خبرگزاری می‌خواهد یک بسته خبری شامل تعدادی خبر را به n نفر از مشترکین خود ارسال کند. هر خبر مورد علاقه تعدادی از مشترکین است. همچنین ارسال هر خبر داخل این بسته خبری دارای هزینه‌ای مشخص است. اگر بدانیم هر یک از مشترکین نهایتا به k خبر علاقه‌مند است روشی k-تقریب و مبتنی بر LP ارائه دهید و در نهایت روش خود را اثبات کنید.

پاسخ سوال:

یک خبرگزاری می‌خواهد یک بسته خبری شامل تعدادی خبر را به تعدادی از n مشترکین خود ارسال کند. هر خبر مورد علاقه تعدادی از مشترکین است و ارسال هر خبر داخل بسته خبری دارای هزینه‌ای مشخص است. همچنین هر مشترک نهایتا به k خبر علاقه‌مند است. باید روشی مبتنی بر LP و با تقریب k ارائه دهیم.

راه حل

این مسأله یک نمونه از مسأله پوشش مجموعه (Set Cover) است که در آن:

هر خبر یک مجموعه است که مشترکین علاقه‌مند به آن را پوشش می‌دهد.

هزینه هر خبر مشخص است.

هدف انتخاب کمترین تعداد خبر با کمترین هزینه کل است به طوری که همه مشترکین حداقل یک خبر مورد علاقه خود را دریافت کنند.

با توجه به اینکه هر مشترک حداکثر به k خبر علاقه‌مند است، می‌توانیم از این ویژگی برای ارائه یک الگوریتم با تقریب k استفاده کنیم.

فرمول‌بندی برنامه‌ریزی خطی (LP):

برای هر خبر j، متغیر تصمیم xj را تعریف می‌کنیم:

xj: احتمال انتخاب خبر j (در LP آسایش‌یافته)

مدل LP به صورت زیر است:

$$minimize \sum(j = 1\ to\ m)\ cj\ *\ xj$$

$$subject\ to:$$

$$\sum(j \in S(i))\ xj\ \geq\ 1 \quad \text{مشترک هر برای } i$$

$$0\ \leq\ xj\ \leq\ 1 \quad \text{خبر هر برای } j$$

که در آن:

cj: هزینه ارسال خبر j

S(i): مجموعه خبرهایی که مشترک i به آن‌ها علاقه‌مند است

الگوریتم تقریبی با ضریب تقریب k:

مدل LP بالا را حل کنیم و مقادیر بهینه x*j را به دست آوریم.

برای هر خبر j، آن را با احتمال $min(1, k \cdot x * j)$ انتخاب کنیم.

اگر پس از این انتخاب، مشترکی بدون خبر باقی ماند، یکی از خبرهای مورد علاقه او را به صورت تصادفی انتخاب کنیم.

شبه‌کد الگوریتم:

الگوریتم انتخاب_خبر:(c, S)

حل LP و به دست آوردن x*

خبرهای_انتخاب_شده = {}

برای هر خبر:j

با احتمال $min(1, k \cdot x * j)$ خبر j را به خبرهای_انتخاب_شده اضافه کن

برای هر مشترک:i

اگر هیچ خبری از S(i) در خبرهای_انتخاب_شده نباشد:

یک خبر j از S(i) را به صورت تصادفی انتخاب کن

خبر j را به خبرهای_انتخاب_شده اضافه کن

برگرداندن خبرهای_انتخاب_شده

اثبات تقریب k:

برای اثبات ضریب تقریب k، باید نشان دهیم:

الگوریتم ما یک پوشش معتبر ارائه می‌دهد (یعنی هر مشترک حداقل یک خبر مورد علاقه خود را دریافت می‌کند).

هزینه مورد انتظار راه حل ما حداکثر k برابر هزینه بهینه LP است.

گام 1: اثبات پوشش کامل

مرحله 3 الگوریتم تضمین می‌کند که هر مشترک حداقل یک خبر مورد علاقه خود را دریافت می‌کند. اما می‌خواهیم نشان دهیم که با احتمال بالا، مرحله 2 به تنهایی پوشش کامل را ارائه می‌دهد.

برای هر مشترک i، احتمال اینکه هیچ خبری از S(i) در مرحله 2 انتخاب نشود:

$P(\text{مشترک } i \text{ پوشش داده نشود}) = \prod(j \in S(i))\ (1 - min(1, k \cdot x * j))$

از آنجایی که $(LP \text{ محدودیت})\ \sum(j \in S(i))\ x * j \geq 1$ و $|S(i)| \leq k$ (هر مشترک حداکثر به k خبر علاقه‌مند است)، می‌توانیم نشان دهیم:

$P(\text{مشترک } i \text{ پوشش داده نشود}) \leq (1 - 1/k)\text{^}k < 1/e$

بنابراین، با احتمال بیشتر از $1 - 1/e$، هر مشترک در مرحله 2 پوشش داده می‌شود. مرحله 3 تضمین می‌کند که همه مشترکین پوشش داده می‌شوند.

گام 2: اثبات ضریب تقریب k

هزینه مورد انتظار الگوریتم ما در مرحله 2:

$$E[\text{هزینه}] = \sum(j = 1\ to\ m)\ cj \cdot min(1, k \cdot xj) \leq k \cdot \sum(j = 1\ to\ m)\ cj \cdot xj$$
$$= k \cdot OPT_LP$$

هزینه اضافی مرحله 3 در بدترین حالت (که همه مشترکین نیاز به پوشش اضافی داشته باشند) حداکثر برابر OPT_LP است.

بنابراین، هزینه کل مورد انتظار الگوریتم ما حداکثر $(k+1)\cdot OPT_LP$ است. اما با تحلیل دقیق‌تر و با استفاده از احتمال پوشش در مرحله 2، می‌توان نشان داد که هزینه مورد انتظار کل حداکثر $k\cdot OPT_LP$ است.

از آنجا که $OPT_LP \leq OPT$ (مقدار بهینه مسأله اصلی)، هزینه مورد انتظار الگوریتم ما حداکثر $k\cdot OPT$ است.

بنابراین، الگوریتم ما یک الگوریتم تقریبی با ضریب تقریب k برای مسأله خبرگزاری است.

سوال 6 – آنتن دهی

صورت سوال:

سوال ۶: آنتن دهی

در منطقه‌ای n دکل برای پوشش آنتن دهی به m مشترک نصب شده است. طبق داده های جمع آوری شده می‌دانیم که هر دکل چه کاربرانی را پوشش می‌دهد. می‌خواهیم k تا از دکل ها را برای تقویت انتخاب کنیم به طوری که بیشتری تعداد کاربران تحت تاثیر قرار بگیرند. روشی با تقریب $1-\frac{1}{e}$ ارائه دهید که به کمک آن این k دکل را انتخاب کنیم. روش خود را اثبات کنید.

راهنمایی: می‌توانید برای اثبات راه حل خود از نامساوی زیر کمک بگیرید:

$$\left(1-\frac{1}{k}\right)^k \leq \frac{1}{e}$$

پاسخ سوال:

در منطقه‌ای n دکل برای پوشش آنتن دهی به مشترک نصب شده است. طبق داده‌های جمع آوری شده می‌دانیم که هر دکل چه کاربرانی را پوشش می‌دهد. می‌خواهیم k تا از دکل‌ها را برای تقویت انتخاب کنیم به طوری که بیشترین تعداد کاربران تحت تأثیر قرار بگیرند. روشی با تقریب $1-1/e$ ارائه دهید که به کمک آن k دکل را انتخاب کنیم.

راه حل

این مسئله یک نمونه کلاسیک از مسئله **پوشش مجموعه با حداکثر پوشش** (Maximum Coverage Problem) است که در آن می‌خواهیم k مجموعه را انتخاب کنیم تا بیشترین تعداد عناصر را پوشش دهیم.

الگوریتم حریصانه (Greedy Algorithm)

برای حل این مسئله از الگوریتم حریصانه استفاده می‌کنیم:

1. در هر مرحله، دکلی را انتخاب می‌کنیم که بیشترین تعداد کاربران جدید (کاربرانی که هنوز توسط دکل‌های انتخاب شده قبلی پوشش داده نشده‌اند) را پوشش دهد.
2. این کار را k بار تکرار می‌کنیم.

پیاده‌سازی الگوریتم

Interactive - shoretest_substring_approximation.py | Max_Covearge.py

D: > Advance Algorithm > HWs > AA_HW6_810103098 > Max_Covearge.py > max_coverage

```python
def max_coverage(towers, users, k):
    """
    towers: لیستی از مجموعه‌ها که هر کدام نشان‌دهنده کاربران تحت پوشش یک دکل است
    users: مجموعه کل کاربران
    k: تعداد دکل‌هایی که می‌توانیم انتخاب کنیم
    """
    selected_towers = []  # دکل‌های انتخاب شده
    covered_users = set()  # کاربران تحت پوشش

    for _ in range(k):
        # پیدا کردن دکلی که بیشترین کاربر جدید را پوشش می‌دهد
        max_additional_coverage = 0
        best_tower = None

        for i, tower in enumerate(towers):
            if i not in selected_towers:
                # محاسبه تعداد کاربران جدیدی که این دکل پوشش می‌دهد
                new_coverage = len(tower - covered_users)
                if new_coverage > max_additional_coverage:
                    max_additional_coverage = new_coverage
                    best_tower = i

        # اگر دکلی پیدا شد که کاربران جدیدی را پوشش می‌دهد
        if best_tower is not None:
            selected_towers.append(best_tower)
            covered_users = covered_users.union(towers[best_tower])

    return selected_towers, covered_users
```

شکل 3- کد پایتون برای الگوریتم **Max Coverage**

اثبات ضریب تقریب

برای اثبات ضریب تقریب $1 - 1/e$ از نامساوی داده شده در صورت سوال استفاده می‌کنیم:

$$1 - (1 - 1/k)^k \geq 1 - 1/e$$

ابتدا چند تعریف می‌کنیم:

- OPT: مجموعه بهینه کاربران که توسط k دکل بهینه پوشش داده می‌شوند.
- |OPT|: تعداد کاربران در راه حل بهینه.
- Si: مجموعه کاربرانی که پس از انتخاب i دکل توسط الگوریتم حریصانه پوشش داده می‌شوند.
- |Si|: تعداد کاربران پوشش داده شده پس از انتخاب i دکل.

فرض کنید در مرحله i هستیم و 1-Si کاربران پوشش داده شده تا این مرحله هستند. حال می‌خواهیم دکل i-ام را انتخاب کنیم.

دکل‌های بهینه (OPT) حداکثر |OPT| کاربر را پوشش می‌دهند. از این k دکل بهینه، حداقل یک دکل وجود دارد که حداقل $\frac{|OPT|-|Si-1|}{k}$ کاربر جدید (که هنوز در $Si - 1$ نیستند) را پوشش می‌دهد.

چون الگوریتم حریصانه در هر مرحله دکلی را انتخاب می‌کند که بیشترین تعداد کاربران جدید را پوشش دهد، پس:

$$| Si | -| Si - 1 |\geq \frac{| OPT | -| Si - 1 |}{k}$$

این نامساوی را می‌توان به صورت زیر بازنویسی کرد:

$$| OPT | -| Si |\leq (1 - 1/k)(| OPT | -| Si - 1 |)$$

با اعمال این نامساوی به صورت بازگشتی:

$$| OPT | -| Sk |\leq (1 - 1/k)^k \cdot| OPT |$$

از آنجا که $S0 = \emptyset$، داریم:

$$\frac{| Sk |}{|OPT|} \geq 1 - (1 - 1/k)^k$$

با استفاده از نامساوی داده شده در صورت مسئله:

$$1 - (1 - 1/k)^k \geq 1 - 1/e$$

بنابراین:

$$\frac{| Sk |}{|OPT|} \geq 1 - 1/e$$

این نشان می‌دهد که الگوریتم حریصانه، راه حلی با ضریب تقریب حداقل $1 - 1/e \approx 0.632$ ارائه می‌دهد.

نتیجه‌گیری

الگوریتم حریصانه ارائه شده، k دکل را انتخاب می‌کند به طوری که حداقل $(1 - 1/e)$ برابر تعداد کاربران در راه حل بهینه را پوشش می‌دهد. این ضریب تقریب بهترین ضریب تقریبی است که می‌توان برای این مسئله NP-hard به دست آورد (مگر اینکه P=NP باشد).

الگوریتم پیشنهادی هم ساده است و هم کارآمد، با پیچیدگی زمانی $O(n \cdot k)$ که n تعداد دکل‌ها و k تعداد دکل‌های انتخابی است.

((پایان))